Expected O(n^1.483)-time girth computation for planar segment intersection graphs

Timothy M. Chan, Yuancheng Yu

doi:10.4230/LIPIcs.SoCG.2026.30; CC BY 4.0

Editorial scope. Complete official sections1–4 and References. Count only the main segment-girth algorithm, including its direct short-cycle preprocessing; do not split clustered distances or support lemmas. Extensions to arbitrary curves mentioned as full-version material are not claimed. A weighted/load planar separator preserves a bounded number of path components while keeping small arbitrary leftovers. The author combines clustered-planar subset distances, bounded-difference min-plus products, and legal alternating walks to recover simple cycles, then balances the two recurrences below the n^(3/2) barrier. All source-specific separator, walk-simplicity, distance and girth proofs are included; the distance subroutine and short-cycle consequence are not split.

Allowed standard prerequisites (editorial).

- Author-cited bounded-difference min-plus product theorem, ordinary BFS and planarization, sparsity of segment intersection graphs with girth above four, existing 3/4-cycle detection algorithms.. Exact original prerequisite statements and references preserved in sections1–4 and References. Source-specific separator Lemma2, legal walks Lemmas3/4, clustered distances Theorem6 and girth Theorem8/Corollary9 remain fully proved within this task.

1 Introduction

Intersection graphs of geometric objects have received considerable attention in computational geometry. They generalize planar graphs (every planar graph is an intersection graph of disks in 2D by the Koebe–Andreev–Thurston theorem), and they arise in numerous applications (for example, unit disk graphs can be used to model communication networks, and intersection graphs of line segments in 2D can be used to model road networks). Over the years, researchers have studied many standard computational problems on geometric intersection graphs, aiming for algorithms that run faster than those for general graphs. The problems considered include the

computation of connected components [1], all-pairs shortest paths [17], diameter [10, 15], minimum cut [12], maximum matching [6], and finding small-sized subgraphs (such as triangles, cycles of a fixed length, cliques of a fixed size, etc.) [14].

In this paper, we focus on another of the most basic graph algorithm problems, which has not been as well explored in the context of geometric intersection graphs: the computation of the girth, i.e., the length of the shortest cycle (see Figure 1). Girth is related to various topics in graph theory, including the chromatic number, treewidth, diameter, Ramsey theory, etc. [5, 22, 39, 27, 18, 23]. It can be computed in $O(mn)$ or $O(n^\omega)$ time for general unweighted undirected graphs with n vertices and m edges [32], where $\omega < 2.371339$ is the matrix multiplication exponent [2]. For the case of planar graphs, a series of papers [24, 25, 13, 46] culminated in a linear-time algorithm by Chang and Lu [19]. However, for intersection graphs, we are aware of only two main algorithmic results:

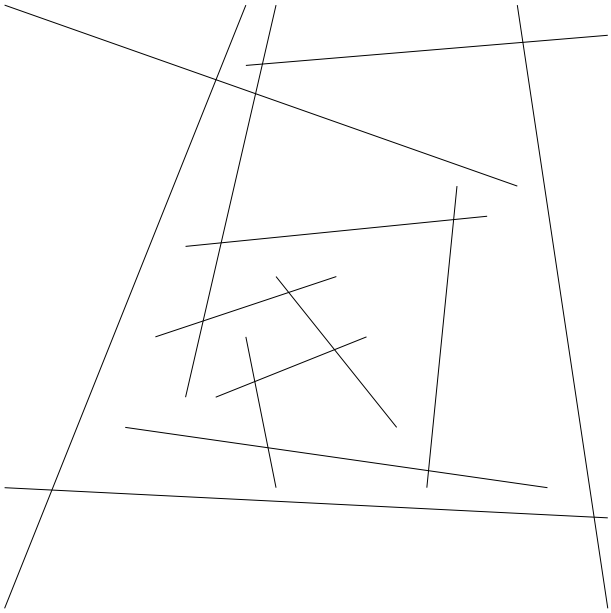


Figure 1: An example of the shortest cycle of a segment intersection graph (segments not in the shortest cycle are dotted). The girth is 6.

1. Kaplan et al. at ESA'19 [33] gave an $O(n \log n)$-time algorithm for computing the girth of the intersection graph of n disks in 2D (in the case of unit disks, this was subsequently improved by Klost and Mulzer [35] to an $O(n)$-time algorithm which does not require the disks be given, just the graph).

2. Chan at SODA'23 [14] gave an $\widetilde{O}(n^{3/2})$-time[1] algorithm for computing the girth of the intersection graph of n line segments in 2D.

[1]The $\widetilde{O}$ notation hides polylogarithmic factors.

Chan's work naturally raises the following question:

Is there an $O(n^{3/2-\varepsilon})$-time algorithm for computing the girth of the intersection graph of n line segments for some constant $\varepsilon > 0$?

The case of line segments is particularly important, not only because of direct relevance to road networks, for example, but also because it serves as a stepping stone towards the general case of algebraic curves or semialgebraic sets in the plane.

1.1 New result

We answer the above open question in the affirmative by presenting the first algorithm that breaks the $n^{3/2}$ barrier. Specifically, our algorithm computes the girth for the intersection graph of n line segments in 2D in $O(n^{1.483})$ expected time. The algorithm also extends to intersection graphs of algebraic curves or semialgebraic sets where each object is connected and has constant description complexity – essentially, most families of constant-size objects in 2D that are commonly encountered.

1.2 Why is the problem challenging?

First note that Kaplan et al.'s approach [33] is not applicable for our problem, as it exploited special properties of disks that do not hold for line segments (namely, that any intersection graph of disks with girth more than 3 must be planar). For the related problem of finding short cycles, Chan [14] extended Kaplan et al.'s result to arbitrary fat objects in any constant dimension, by using a shifted quadtree approach, but line segments are not fat.

For intersection graphs of line segments, Chan [14] has already given an $O(n^{1.408})$-time algorithm for detecting a triangle (i.e., 3-cycle), and an $\widetilde{O}(n)$-time algorithm for detecting a 4-cycle. Thus, it suffices to handle the remaining case when the girth is greater than 4. It is known that any intersection graph of n line segments without 4-cycles must be sparse, i.e., has $O(n)$ edges. We can explicitly construct the arrangement of the line segments and form a planar graph of $O(n)$ size whose vertices are the endpoints and intersection points. This way, our problem is reduced to a variant of the girth problem on a planar graph, where the cost of a cycle is the number of turns – turning at an intersection point has unit cost, but going straight through an intersection has zero cost.

However, this variant does not seem directly reducible to the standard girth problem on planar graphs. One could create a gadget at each intersection point and obtain a graph with $0/1$ edge weights, such that the weight of a path in the new graph corresponds to the number of turns along the path. But the resulting graph would no longer be planar (it would be 1-planar instead); besides, unintended small cycles would be introduced in such a gadget. In addition, many properties about shortest paths in planar graphs no longer hold. For example, two shortest paths in a planar graph cannot cross more than once (if ties are broken carefully), but two minimum-turn paths may cross numerous times. Consequently, many techniques devised for shortest-path-related problems in planar graphs (e.g., based on some kind of Monge property [29], or abstract Voronoi diagrams [11,

30], or VC-dimension arguments [37, 26, 36, 15]) seem to fall apart for our problem.

One technique that still works is separators. In fact, Chan's previous $\widetilde{O}(n^{3/2})$-time algorithm is obtained by a simple application of separators. We can divide the graph into two parts by a separator of size $O(\sqrt{n})$, and recurse on both parts. This handles the case when the shortest cycle lies completely in one part. To deal with the remaining case when the shortest cycle passes through a separator vertex, we can run BFS from each of the $O(\sqrt{n})$ separator vertices, in $\widetilde{O}(n^{3/2})$ total time.

For the case when the girth is a constant (or for detecting a cycle of a fixed constant length), Chan [14] has shown how to adapt the separator approach to obtain a faster algorithm running in $\widetilde{O}(n^{\omega/2}) \leq O(n^{1.186})$ time. The main observation is that the $O(\sqrt{n})$ calls to BFS is overkill – we only need to compute shortest path distances with both sources and destinations from the $O(\sqrt{n})$ separator vertices. We can compute such distances recursively as we compute the girth. Combining the output of the two recursive calls involves computing min-plus products of $O(\sqrt{n}) \times O(\sqrt{n})$ distance matrices. When the matrix entries are integers bounded by a constant, min-plus product reduces to $O(1)$ number of standard (Boolean) matrix product. This is roughly how the $\widetilde{O}(n^{\omega/2})$ time bound was obtained.

For larger girth, however, this approach does not work well, because the complexity of min-plus product grows for larger integer values. (Another issue is the use of color-coding [3] in Chan's cycle detection algorithm [14], which led to exponential dependence on the cycle length.)

On the other hand, when the girth g is very large, one could obtain an $\widetilde{O}(n^2/g)$-time algorithm by a well-known hitting set technique [44, 47] (there exists a subset of $\widetilde{O}(n/g)$ vertices which hit all shortest paths of length $\Omega(g)$). The main difficulty lies in the intermediate case, when the girth is not too small and not too large.

1.3 Our approach

Ingredient 1: Bounded-difference min-plus matrix multiplication.

The previous separator-based approach faces an $n^{3/2}$ barrier because min-plus matrix multiplication is widely conjectured to require near-cubic time (according to the "All-Pairs Shortest Paths (APSP) hypothesis" from fine-grained complexity [45]). Our first idea is to employ recent results which solve certain powerful special cases of min-plus product in truly subcubic time – namely, the case when the input matrices are integer-valued and satisfy a bounded-difference property, i.e., the difference between any two adjacent entries (in each row or in each column) is at most a constant. Chan and Lewenstein [16] gave the first truly subquadratic algorithm for the related min-plus convolution problem for bounded-difference sequences, while Bringmann et al. [9] gave the first truly subcubic algorithm for bounded-difference min-plus product. Bringmann et al.'s original algorithm ran in $O(n^{2.8244})$ time, and after a series of improvements

[31, 20], Chi et al. [21] obtained the current best running time of $\widetilde{O}(n^{(3+\omega)/2}) \leq O(n^{2.686})$ (see Section 2.1).

Bounded-difference min-plus convolution and matrix multiplication have recently been applied to solve a variety of problems from different domains, including language edit distance and RNA folding [9], single-source replacement paths and Range Mode [31], tree edit distance [40], approximate APSP [43], knapsack [7, 8], etc. Our algorithm, being one of the first applications to a computational geometry problem, is thus noteworthy.

Ingredient 2: A variant of the planar-graph separator theorem.

Why is the bounded-difference property relevant in our application? Suppose the separator S is connected, e.g., is a cycle (as in Miller's version of the separator theorem [41]). Then for any vertex q and two adjacent separator vertices $s, s' \in S$, $d(s, q)$ and $d(s', q)$ differ by at most 1 due to the triangle inequality. Thus, the distance matrices we encounter during recursion would indeed satisfy the bounded-difference property.

Unfortunately, cycle separators do not always exist when the given planar graph is not 2-connected. Our key idea is to use an interesting variant of the planar-graph separator theorem, which holds for any planar graphs: given any $\alpha > 0$, there always exists a separator S consisting of two subsets P and Q, where

- the first subset P has size $O(n^{1/2+\alpha})$ but has at most 2 connected components (in fact, at most 2 paths);
- the second subset Q may not be connected but has size $O(n^{1/2-\alpha})$.

We are not aware of this variant of the separator theorem explicitly stated before (though we will not be surprised if it appeared before), but it follows by straightforward modification of standard proofs. See Section 2.2 for the precise statement and proof.

This variant of separators is exactly the tool we need to break the $n^{3/2}$ barrier: since P has only $O(1)$ connected pieces, we can use bounded-difference min-plus products to compute distances involving P faster; on the other hand, since Q has size $\ll \sqrt{n}$, we can afford to compute distances involving Q more naively. We choose the trade-off parameter α carefully to balance cost. This is the main high-level idea of our algorithm. Quite a bit of technical effort, however, is needed to make the idea work (for example, the number of connected components may grow as we recurse, but not by much, as we will show, and we eventually need two separate recursive algorithms, one for computing distances and one for computing the girth). See Sections 3–4 for the detailed algorithm and analysis.

Ingredient 3: Ensuring simplicity of walks.

When computing a shortest path between two vertices, we do not need to worry about ensuring simplicity of the path, since a shortest walk must automatically be a shortest path (if a vertex is repeated,

we can take a short-cut to obtain a shorter walk). However, more care is needed for the shortest cycle problem in undirected graphs. When concatenating several shortest paths to form a candidate cycle (via min-plus products of several distance matrices), one potential issue is that the resulting closed walk might become trivial (we might be completely retracing our steps, so that short-cutting yields an empty walk!). The earlier algorithm by Chan [14] for detecting constant-length cycles used the color-coding technique [3] to ensure simplicity, but unfortunately color-coding yields exponential dependence on the cycle length. For general graphs, there are other strategies to ensure simplicity, (e.g., by computing shortest paths in different subgraphs with some vertices and edges removed, or with directions added, so as to avoid concatenating two identical shortest paths). But ensuring simplicity is trickier in our setting, since we need to retain the bounded-difference property if we work with new subgraphs. Still, we have found one way to deal with this technical issue, using ideas given in Section 2.3.

2 Preliminaries

Before presenting our algorithm, we begin by providing more details on the three ingredients mentioned in the introduction.

Let $\delta_G(u,v)$ denote the shortest-path length between u and v in an unweighted undirected graph G. Let $\delta_G[A,B]$ denote the distance matrix $\{\delta_G(u,v)\}_{u\in A, v\in B}$ for two vertex subsets A and B in a graph G. Let $G[S]$ denote the subgraph of G induced by a vertex subset S.

2.1 Bounded-difference min-plus product

Given two matrices A, B, their min-plus product $A\star B$ is defined as

$$(A\star B)(i,j) = \min_k (A(i,k) + B(k,j)).$$

As mentioned, we will use a known subcubic algorithm for computing the min-plus product of two $n\times n$ integer matrices that satisfy a bounded-difference property. The following is due to Chi et al. [21]:

Lemma 1.

Given two integer $n\times n$ matrices A and B where each column of A satisfies the bounded-difference property (i.e., $|A(i,j) - A(i+1,j)| \leq O(1)$ for every i,j), we can compute their min-plus product $A\star B$ in $\widetilde{O}(n^{(3+\omega)/2}) \leq O(n^{2.686})$ expected time.

The same holds if instead each row of A satisfies the bounded-difference property (i.e., $|A(i,j) - A(i,j+1)| \leq O(1)$), or if the property holds for B rather than A.

2.2 A variant of the planar separator theorem

The second ingredient is an interesting variant of the planar-graph separator theorem, in which the separator is divided into two subsets P and Q: P may be larger but has only 2 connected pieces, while Q may not be connected but is smaller. The precise statement is given below (the original theorem corresponds to the case when $p=\sqrt{n}$ with unit loads, while our application requires the case when

$p \gg \sqrt{n}$). The proof follows by modifying standard proofs [38, 34, 28].

Lemma 2.
Let $G = (V, E)$ be a planar graph with n vertices, where each vertex has a nonnegative weight and a nonnegative load, with total weight W, total load Λ, and the maximum individual vertex weight is at most εW for a sufficiently small constant $\varepsilon > 0$. For a given parameter p, we can partition V into subsets V_1, V_2, S, in $O(n)$ time, such that

- the total weight of V_i is at most $3W/4$ for each $i \in \{1, 2\}$;
- there are no edges in $V_1 \times V_2$;
- S can be further partitioned into P and Q such that P is the union of at most two paths of length $O(p)$, and Q has total load at most $O(\Lambda/p)$.

Proof.
We first assume that G is connected. Let $\mathcal{T}$ be a BFS tree of G from a fixed source vertex. For any $uv \notin \mathcal{T}$, let cycle$(\mathcal{T}, uv)$ denote the unique cycle in $\mathcal{T} \cup \{uv\}$ (called the fundamental cycle), which consists of uv and the unique path in $\mathcal{T}$ connecting u and v.

Let Σ be a (combinatorial) triangulation of $\mathcal{T}$. Define face weights of Σ by examining each vertex v, picking an arbitrary incident face f, and adding v's weight to f. Then the total face weight is W. Consider the dual tree with respect to $\mathcal{T}$ in Σ (which has maximum degree 3). By standard results on tree separators [34, 28]. we can find an edge which separates this dual tree into two parts each with total face weight at most $3W/4$. In Σ, this corresponds to an edge uv such that either side of $C =$ cycle$(\mathcal{T}, uv)$ has total face weight at most $3W/4$; this also bounds the total weight of the vertices strictly inside/outside C. If $|C| \le p$ then the desired separator can obtained by setting $P = C$ and $Q = \emptyset$. Below we may assume $|C| > p$.

Let x be the lowest common ancestor of u, v in $\mathcal{T}$, and $\mathcal{T}'$ be the subtree of $\mathcal{T}$. Let $L_i \subset V$ be the set of vertices in level i of $\mathcal{T}'$ (with $L_0 = \emptyset$). Let m be the smallest integer such that the total weight of vertices in the first m levels of $\mathcal{T}'$ is at least $W/2$. By the pigeonhole principle, there exists $\ell \in [m-p, m)$ such that L_ℓ has total load at most Λ/p. Similarly there exists $h \in (m, m+p]$ such that L_h has total load at most Λ/p.

If $V_{\le \ell} = \bigcup_{i=1}^{\ell} L_i$ has weight at least $W/4$, then since $V_{\le \ell}$ has weight at most $W/2$ by definition of m, the desired separator can be obtained by setting $Q = L_\ell$ and $P = \emptyset$.

Similarly, if $V_{\ge h} = \bigcup_{i=h}^{\infty} L_i$ has weight at least $W/4$, then the desired separator can be obtained by setting $Q = L_h$ and $P = \emptyset$.

In the remaining case, $V_m = \bigcup_{i=\ell+1}^{h-1} L_i$ has weight at least $W/2$. Let $V_{\text{in}}, V_{\text{out}}$ be the set of vertices in V_m that are strictly inside or outside C, respectively. Without loss of generality, say V_{in} has weight at least $W/4$. Recall that V_{in} has weight at most $3W/4$ by definition of C. Let P_u (resp. P_v) be the portion of the shortest path from x to u (resp.

v) with levels between ℓ and h. Then P_u and P_v are paths of length $O(p)$. The desired separator can be obtained by setting $P = P_u \cup P_v$ and $Q = L_\ell \cup L_h$. See Figure 2.

Finally, we consider the general case when G may have multiple connected components. If all components have weight at most $W/2$, a trivial separator exists with $S = \emptyset$. On the other hand, if G has one component of weight at least $W/2$, we can apply the lemma to the largest-weight component and add the remaining components to the smaller of the two subsets V_1, V_2 in terms of weight (which would still have total weight at most $3W/4$). ◀

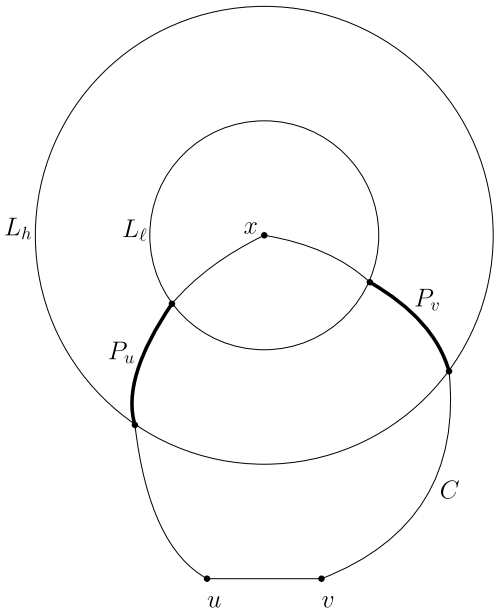


Figure 2: Proof of Lemma 2. Solid curves consist of edges in G, whereas dashed curves may contain edges in the triangulation but not in G.

2.3 Ensuring simplicity of walks

In this subsection, we formulate a type of legal alternating closed walks so that a shortest closed walk of this type is automatically a simple cycle. This will be useful later in Section 4. (Warning: both the formulation and the proof of correctness below are somewhat subtle.)

In the following, we imagine that a separator has been computed, and the two parts are colored red and blue, while the separator vertices are colored white.

Lemma 3.

Let G be an undirected graph, where each vertex is colored red, blue, or white, and there are no edges between red and blue vertices. Consider a closed walk C which is the concatenation of simple paths $\pi_0^R, \tau_0, \pi_0^B, \tau_0', \ldots, \pi_{k-1}^R, \tau_{k-1}, \pi_{k-1}^B, \tau_{k-1}'$ with $k \geq 1$, where each π_i^R consists of red and white vertices only and has at least one red vertex, each

π_i^B consists of blue and white vertices only and has at least one blue vertex, and each τ_i and τ_i' consist of white vertices only (but may be empty). Then C must contain a nontrivial simple cycle.

Proof.

Suppose that two consecutive edges of C are identical, i.e., we see 3 consecutive vertices u, v, u in C. If uv is the last edge of π_i^R and vu is the first edge of τ_i, then u and v are white vertices and we can remove v from C and still have a closed walk satisfying the stated condition. If uv is the last edge of π_i^R, and τ_i is empty, and vu is the first edge of π_i^B, then u and v must again be white vertices and we can remove v from C and still have a closed walk satisfying the stated condition. After handling all other similar cases, we may assume that no two consecutive edges of C are identical. Now, C forms a subgraph where every vertex has degree at least 2, and so must contain a nontrivial simple cycle. ◀

Lemma 4.

Let G be an undirected graph, where each vertex is colored red, blue, or white, and there are no edges between red and blue vertices. Let R, B, and W be the subset of red, blue, and white vertices respectively. Define the following:

- A simple path π from u to v in G is a legal red path if π lies in $G[R \cup W]$, $u, v \in W$, and $\delta_{G[R \cup W]}(u, v) \neq \delta_{G[W]}(u, v)$.
- A simple path π from u to v in G is a legal blue path if π lies in $G[B \cup W]$, $u, v \in W$, and $\delta_{G[B \cup W]}(u, v) \neq \delta_{G[W]}(u, v)$.
- A closed walk C is a legal alternating walk if it is the concatenation of simple paths $\pi_0^R, \tau_0, \pi_0^B, \tau_0', \ldots, \pi_{k-1}^R, \tau_{k-1}, \pi_{k-1}^B, \tau_{k-1}'$ with $k \geq 1$, where each π_i^R is a legal red path, each π_i^B is a legal blue path, and each τ_i and τ_i' lie in $G[W]$.

Then

(i) the girth of G is at most the length of a shortest legal alternating closed walk;

(ii) furthermore, equality holds if the girth of G is not equal of girth of $G[R \cup W]$ or the girth of $G[B \cup W]$.

Proof.

For (i), consider a shortest legal alternating closed walk C. Then C is the concatenation of simple paths $\pi_0^R, \tau_0, \pi_0^B, \tau_0', \ldots, \pi_{k-1}^R, \tau_{k-1}, \pi_{k-1}^B, \tau_{k-1}'$ with $k \geq 1$, where each π_i^R is a legal red path, each π_i^B is a legal blue path, and each τ_i and τ_i' lie in $G[W]$. Each π_i^R must pass through at least one red vertex, because $\delta_{G[R \cup W]}(u(\pi_i^R), v(\pi_i^R)) \neq \delta_{G[W]}(u(\pi_i^R), v(\pi_i^R))$, where $u(\pi)$ and $v(\pi)$ denote the starting and ending vertices of a path π. Similarly, each π_i^B must pass through at least one blue vertex. So, the condition in Lemma

3 is satisfied, and C must contain a nontrivial simple cycle. Hence, the girth of G is at most the length of C.

For (ii), consider a shortest simple cycle C in G, of length g. If there are multiple shortest simple cycles, pick one with the minimum number of non-white vertices. We can express C as a concatenation of simple paths $\pi_0^R, \pi_0^B, \ldots, \pi_{k-1}^R, \pi_{k-1}^B$ with $k \geq 1$, where each π_i^R lies in $G[R \cup W]$, uses at least one red vertex, and starts and ends with white vertices, and each π_i^B lies in $G[B \cup W]$, uses at least one blue vertex, and starts and ends with white vertices. (Note that C contains at least one red and at least one blue vertex, by the assumption in stated in (ii).) If $\delta_{G[R\cup W]}(u(\pi_i^R), v(\pi_i^R)) = \delta_{G[W]}(u(\pi_i^R), v(\pi_i^R))$ for some i, then we can replace π_i^R with a shortest path γ from $u(\pi_i^R)$ to $v(\pi_i^R)$ in $G[W]$, and obtain a closed walk which is the concatenation of two non-identical simple paths γ and $C \setminus \pi_i^R$, and so must contain a nontrivial simple cycle; the cycle has length at most g and uses fewer non-white vertices: a contradiction. Thus, each π_i^R is a legal red path. Similarly, each π_i^B is a legal blue path. Hence, C is a legal alternating closed walk. ◀

(An astute reader might wonder why we have overcomplicated the definition of legal alternating walk by including the white subpaths τ_i, since part (ii) of the proof of Lemma 4 does not require them. The purpose for including them is computational, as we will see later: without them, intermediate distance matrices may no longer satisfy the bounded-difference property.)

3 Multiple Distances in Clustered Planar Graphs

As a warm-up, we first present an algorithm for a subproblem (of independent interest), on computing all distances with sources and destinations from a given subset X, in an intersection graph of line segments. It is actually more convenient to solve the problem for a more general family of graphs, which we call clustered planar graphs, where each vertex in a planar graph is replaced by a "cluster" of c vertices:

Definition 5.
A c-clustered planar graph refers to the product graph $\widehat{G} = G \times K_c$ where $G = (V, E)$ is a planar graph and K_c is the c-clique. In other words, the vertices of $\widehat{G}$ are pairs $(v, j) \in V \times \{1, \ldots, c\}$, and there is an edge between two vertices (u, i) and (v, j) whenever $uv \in E$ or $u = v$.

Known shortest-path-related algorithms for planar graphs do not necessarily generalize to clustered planar graphs (because of the lack of Monge property, VC-dimension arguments, etc.). We prove the following result, which beats the obvious $\widetilde{O}(xn)$ time bound, for example, when x is around $\sqrt{n}$:

Theorem 6.
Let $\widehat{G} = G \times K_c$ be a c-clustered planar graph, where $G = (V, E)$ has n vertices and $c = O(1)$. Suppose each edge of $\widehat{G}$ has a nonnegative integer weight bounded by $O(1)$. Given a subset X of x vertices in G, we can compute $\delta_{\widehat{G}}[X \times K_c, X \times K_c]$ in $\widetilde{O}(n^{3/2-\alpha} + n^{(1/2+\alpha)(3+\omega)/2} + x^{(3+\omega)/2} + x^2 n^{1/2-\alpha})$ expected time for any given α.

Proof.

We will solve an extended problem: given G, $\widehat{G}$, X, and an additional subset B of β vertices in G such that B has γ components in G, we want to compute $\delta_{\widehat{G}}[(B \cup X) \times K_c, (B \cup X) \times K_c]$. (Initially, $B = \emptyset$ and $\beta = \gamma = 0$.)

Step 1: Build a separator.

Give each vertex in $V \setminus B$ weight 1 and each vertex in B weight $1+w$ for a parameter w (assuming $n+w\beta \gg (1+w)/\varepsilon$); the total weight is $n + w\beta$. Give each vertex in $V \setminus B$ load 1 and each vertex in B load n/β; the total load is $O(n)$. We apply Lemma 2 for a value p to be set later, to partition V into V_1, V_2, S such that

- $|V_i| + w|V_i \cap B| \leq 3(n + w\beta)/4$ for each $i \in \{1, 2\}$;
- there are no edges of G in $V_1 \times V_2$;
- S can be further partitioned into P and Q such that P is the union of at most two paths in G of length $O(p)$, and Q has total load at most $O(n/p)$ – this implies that $|Q| \leq O(n/p)$ and $|Q \cap B| \leq O(\frac{n/p}{n/\beta}) = O(\beta/p)$ simultaneously.

Let $\widehat{V} = V \times K_c$, $\widehat{S} = S \times K_c$, $\widehat{P} = P \times K_c$, $\widehat{Q} = Q \times K_c$, $\widehat{B} = B \times K_c$ and $\widehat{X} = X \times K_c$.

Step 2: Recurse.

For each $i \in \{1, 2\}$, we recursively solve the problem for $G_i = G[V_i \cup S]$, $\widehat{G}_i = G_i \times K_c$ (with edge weights inherited from $\widehat{G}$), $B_i = (V_i \cap B) \cup P \cup (Q \cap B)$, and $X_i = V_i \cap X$.

Let $n_i = |V_i \cup S|$, $\beta_i = |B_i|$, γ_i be the number of components of B_i in G_i, and $x_i = |X_i|$. Then $n_1 + n_2 \leq n + O(p + n/p)$, $\beta_1 + \beta_2 \leq \beta + O(p + \beta/p)$, $n_i + w\beta_i \leq 3(n + w\beta)/4 + O(w(p + n/p))$, and $x_1 + x_2 \leq x$.

To bound γ_i, consider a component C of B_i in G_i. If C does not visit S, then C is an original component of B (since $S \cap B \subseteq B_i$). If C visits a vertex of P, then C contains one of the two paths in P (since $P \subseteq B_i$). If C visits a vertex of $Q \cap B$, then C can be charged to that vertex. It follows that $\gamma_1 + \gamma_2 \leq \gamma + O(1 + \beta/p)$.

Step 3: Compute $\delta_{\widehat{G}}[\widehat{B} \cup \widehat{X}, \widehat{B} \cup \widehat{X}]$.

We now compute $\delta_{\widehat{G}}[\widehat{B} \cup \widehat{X}, \widehat{B} \cup \widehat{X}]$ by the following expression:

$$\begin{aligned}
&\delta_{\widehat{G}_1}[\widehat{B} \cup \widehat{X}, \widehat{B} \cup \widehat{X}] \vee \delta_{\widehat{G}_2}[\widehat{B} \cup \widehat{X}, \widehat{B} \cup \widehat{X}] \vee \\
&(\delta_{\widehat{G}}[\widehat{B} \cup \widehat{X}, \widehat{Q}] \star \delta_{\widehat{G}}[\widehat{Q}, \widehat{B} \cup \widehat{X}]) \vee \\
\Big((&\delta_{\widehat{G}_1}[\widehat{B} \cup \widehat{X}, \widehat{P}] \vee \delta_{\widehat{G}_2}[\widehat{B} \cup \widehat{X}, \widehat{P}]) \star (\delta_{\widehat{G}_1}[\widehat{P}, \widehat{P}] \star \delta_{\widehat{G}_2}[\widehat{P}, \widehat{P}])^n \star \\
&(\delta_{\widehat{G}_1}[\widehat{P}, \widehat{B} \cup \widehat{X}] \vee \delta_{\widehat{G}_2}[\widehat{P}, \widehat{B} \cup \widehat{X}])\Big),
\end{aligned}$$

where $M \vee M'$ denotes the entry-wise minimum of two matrices M and M', and M^k denotes the k-th power under min-plus product.

The expression is correct because for $\widehat{u}, \widehat{v} \in \widehat{B} \cup \widehat{X}$, if a shortest path π from $\widehat{u}$ to $\widehat{v}$ in $\widehat{G}$ avoids $\widehat{S}$, then its length is $\delta_{\widehat{G}_1}(\widehat{u}, \widehat{v})$ or $\delta_{\widehat{G}_2}(\widehat{u}, \widehat{v})$; otherwise, if π passes through $\widehat{Q}$, then its length is $(\delta_{\widehat{G}}[\widehat{B} \cup \widehat{X}, \widehat{Q}] \star \delta_{\widehat{G}}[\widehat{Q}, \widehat{B} \cup \widehat{X}])(\widehat{u}, \widehat{v})$; otherwise, if π passes through $\widehat{P}$, starting from $\widehat{G}_i$ and ending in $\widehat{G}_j$, then its length is $(\delta_{\widehat{G}_1}[\widehat{B} \cup \widehat{X}, \widehat{P}] \star (\delta_{\widehat{G}_1}[\widehat{P}, \widehat{P}] \star \delta_{\widehat{G}_2}[\widehat{P}, \widehat{P}])^n \star \delta_{\widehat{G}_j}[\widehat{P}, \widehat{B} \cup \widehat{X}])(\widehat{u}, \widehat{v})$.

To evaluate the expression, we first compute $\delta_{\widehat{G}}[\widehat{V}, \widehat{Q}]$ and $\delta_{\widehat{G}_i}[\widehat{V}, \widehat{Q}]$ for each $i \in \{1, 2\}$ by running BFS from each vertex of $\widehat{Q}$. Since $|Q| = O(n/p)$, this part takes $\widetilde{O}(n^2/p)$ time.

The matrix $\delta_{\widehat{G}_i}[\widehat{B} \cup \widehat{X}, \widehat{B} \cup \widehat{X}]$ is available, since $(B \cup X) \cap (V_i \cup S) \subseteq B_i \cup X_i \cup Q$, and we know $\delta_{\widehat{G}_i}[\widehat{B}_i \cup \widehat{X}_i, \widehat{B}_i \cup \widehat{X}_i]$ from the two recursive calls, and we already know $\delta_{\widehat{G}_i}[\widehat{V}, \widehat{Q}]$.

The matrices $\delta_{\widehat{G}_i}[\widehat{B} \cup \widehat{X}, \widehat{P}]$ and $\delta_{\widehat{G}_i}[\widehat{P}, \widehat{P}]$ are also available, since $P \subseteq B_i$.

We now describe how to compute $\delta_{\widehat{G}}[\widehat{B} \cup \widehat{X}, \widehat{Q}] \star \delta_{\widehat{G}}[\widehat{Q}, \widehat{B} \cup \widehat{X}]$ in (2). For any component $\widehat{C} \subseteq \widehat{B}$, the columns of $\delta_{\widehat{G}}[\widehat{C}, \widehat{Q}]$ satisfy the bounded-difference property, after reordering the rows according to an Euler tour of a spanning tree of C – some rows may need to be duplicated, but the number of rows increases by only a constant factor. (Note that all vertices in a common cluster $\{v\} \times K_c$ are within a constant distance from each other.) Since B has γ components, we can compute $\delta_{\widehat{G}}[\widehat{B}, \widehat{Q}] \star \delta_{\widehat{G}}[\widehat{Q}, \widehat{B} \cup \widehat{X}]$ by Lemma 1 in $\widetilde{O}(\gamma(\beta + n/p + x)^{(3+\omega)/2})$ time. The product $\delta_{\widehat{G}}[\widehat{B} \cup \widehat{X}, \widehat{Q}] \star \delta_{\widehat{G}}[\widehat{Q}, \widehat{B}]$ is symmetric. It remains to compute $\delta_{\widehat{G}}[\widehat{X}, \widehat{Q}] \star \delta_{\widehat{G}}[\widehat{Q}, \widehat{X}]$: here, since we don't have the bounded-difference property, we naively compute the product in $O(|X|^2|Q|) = O(x^2 n/p)$ time.

Finally, we describe how to compute (3). The columns of $\delta_{\widehat{G}_i}[\widehat{P}, \widehat{P}]$ satisfy the bounded-difference property, after dividing the matrix into two parts and reordering the rows, since P is the union of at most two walks; so, we can compute $\delta_{\widehat{G}_1}[\widehat{P}, \widehat{P}] \star \delta_{\widehat{G}_2}[\widehat{P}, \widehat{P}]$ in $\widetilde{O}(p^{(3+\omega)/2})$ time. We can compute the n-th power by repeated squaring using $O(\log n)$ bounded-difference matrix products. (Note that if the columns of a matrix M satisfy the bounded-difference property, then the columns of a product $M \star M'$ also satisfy the bounded-difference property.)

The rows of $\delta_{\widehat{G}_i}[\widehat{B} \cup \widehat{X}, \widehat{P}]$ satisfy the bounded-difference property (after dividing the matrix into two parts), and so the other products can be computed in a similar way.

Running time.

The time complexity $T(n, \beta, \gamma, x)$ of the algorithm satisfies the following recurrence:

$$T(n,\beta,\gamma,x) \le \max_{\substack{n_1,n_2,\beta_1,\beta_2,x_1,x_2:\\ n_i+w\beta_i\le 3(n+w\beta)/4+O(w(p+n/p)),\\ n_1+n_2\le n+O(p+n/p),\\ \beta_1+\beta_2\le\beta+O(p+\beta/p),\\ \gamma_1+\gamma_2\le\gamma+O(1+\beta/p),\\ x_1+x_2\le x}} \Big(T(n_1,\beta_1,\gamma_1,x_1)+T(n_2,\beta_2,\gamma_2,x_2)+ \widetilde{O}(n+n^2/p+ \gamma(\beta+p+n/p+x)^{(3+\omega)/2}+x^2n/p)\Big).$$

Solving the recurrence.

Intuitively, one could choose $p \approx n^{1/2+\alpha}$, and think of β as $\widetilde{O}(p)$ and γ as $\widetilde{O}(1)$, and see that the cost is upper bounded by $n^{3/2-\alpha}+x^{(3+\omega)/2}+x^2n^{1/2-\alpha}$ for any sufficiently small positive α (as long as $\omega<3$), at least at the top few levels of the recursion.

Formally, let r be a fixed parameter, to be set later, with $r \gg w(p+n/p)\log n$. For a node u in the recursion tree, let n_u,β_u,γ_u,X_u denote its input parameters. Suppose we stop the recursion at a node u whenever $n_u+w\beta_u \le r$; in particular, this condition implies $n_u \le r$ and $\beta_u \le r/w$ simultaneously. Then the parent v of each leaf satisfies $\mu_v+w\beta_v > r$. This recursion subtree has depth at most $O(\log \frac{n+w\beta}{r})$. Let ξ be the size of this recursion subtree. Then $\sum_v(n_v+w\beta_v) \le O((n+w\beta+\xi\cdot w(p+n/p))\log\frac{n+w\beta}{r})$, where the summation is over each v that is the parent of some leaf. Thus, $\xi \le O(\frac{1}{r}\sum_v(n_v+w\beta_v)) \le O(\frac{n+w\beta}{r}\log\frac{n+w\beta}{r})+o(\xi)$, implying that $\xi \le O(\frac{n+w\beta}{r}\log\frac{n+w\beta}{r})$.

This leads to a new recurrence:

$$T(n,\beta,\gamma,x) \le \max_{\substack{x_1,x_2,\ldots:\\ \sum_k x_k\le x}} \sum_{k=1}^{O(\frac{n+w\beta}{r}\log\frac{n+w\beta}{r})} \big(T(r,r/w,\gamma+\widetilde{O}(1+\beta/p),x_k)+ \widetilde{O}\big(n+n^2/p+ \gamma(\beta+p+n/p+x)^{(3+\omega)/2}+x^2n/p\big)\big).$$

We now set $\beta=n^{1/2+\alpha}$, $w=n^{1/2-\alpha}$, $r=n/t$, and $p=n^{1/2+\alpha}/\log^{1+\varepsilon}n$ for a sufficiently large constant t. Note that indeed $r \gg w(p+n/p)\log n$. By loosely bounding r/w by $(n/t)^{1/2+\alpha}$, the recurrence becomes:

$$T(n,n^{1/2+\alpha},\gamma,x) \le \max_{\substack{x_1,x_2,\ldots:\\ \sum_k x_k\le x}} \sum_{k=1}^{O(t\log t)} \big(T(n/t,(n/t)^{1/2+\alpha},\gamma+\widetilde{O}(1),x_k)+ \widetilde{O}(n^{3/2-\alpha}+\gamma(n^{(1/2+\alpha)(3+\omega)/2}+x^{(3+\omega)/2}) x^2n^{1/2-\alpha})\big).$$

This solves to $T(n,n^{1/2+\alpha},\gamma,x) = \widetilde{O}(n^{3/2-\alpha}+(1+\gamma)(n^{(1/2+\alpha)(3+\omega)/2}+x^{(3+\omega)/2})+x^2n^{1/2-\alpha})$. ◀

Corollary 7.

Given a set A of line segments in $\mathbb{R}^2$ and a subset $X \subset A$ of x segments, we can compute $\delta_G[X,X]$, where G is the intersection graph

of A, in $\widetilde{O}(\mu^{3/2-\alpha} + \mu^{(1/2+\alpha)(3+\omega)/2} + x^{(3+\omega)/2} + x^2\mu^{1/2-\alpha})$ expected time for any given α, where μ is the number of endpoints and intersection points of A.

Proof.

For simplicity, we assume that the given segments are in general position (e.g., no 3 segments intersect at a common point).

First we compute the intersections of the line segments in A, for example, by applying Bentley and Ottmann's plane-sweep algorithm [4], which runs in $O(\mu \log \mu)$ time.

Let H be the planarized intersection graph of A, i.e., the planar graph formed by the arrangement of A, where the vertices consist of the segment endpoints and intersection points of A, and the edges of H correspond to pairs of adjacent vertices in the arrangement.

Let $\widehat{H}$ be the clustered planar graph $H \times K_4$ (see Figure 3). Number the edges incident to each vertex of H from $\{1, 2, 3, 4\}$ in clockwise order. Give the edge $(u, i)(v, j)$ weight 0 in $\widehat{H}$ if uv is in H and the i-th edge incident to u coincides with the j-th edge incident to v. In addition, for each vertex u of degree 4 in H, give the edges $(u, 1)(u, 2)$, $(u, 2)(u, 3)$, $(u, 3)(u, 4)$, $(u, 4)(u, 1)$ weight 1, and give the edges $(u, 1)(u, 3)$ and $(u, 2)(u, 4)$ weight 0. Give all other edges weight 2 in $\widehat{H}$. Let $u(s)$ denote the left endpoint of a segment s. Then for any two segments s and t, the distance between s and t in G is equal to the distance between $(u(s), 1)$ and $(u(t), 1)$ in $\widehat{H}$. So, the result immediately follows by applying Theorem 6 to $\widehat{H}$ and $\{u(s) : s \in X\}$. ◀

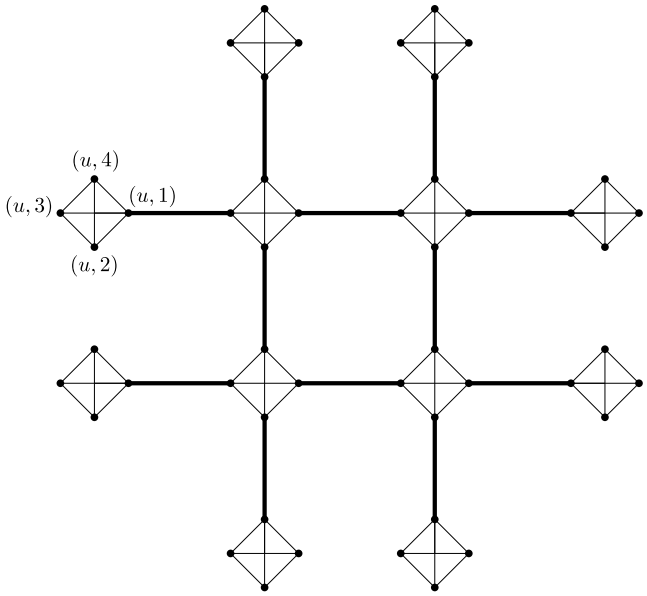


Figure 3: The clustered planar graph $\widehat{H}$. The dotted edges and the dashed edges have weight 0, while the solid edges have weight 1. Edges with weight 2 are omitted.

4 Girth for Line Segments

We now present our girth algorithm for intersection graphs of line segments. It is tempting to continue to work within the clustered planar graph framework, but unfortunately a clustered planar graph by definition contains short cycles in every cluster (furthermore, when generalizing from segments to curves later, an additional difficulty is that simple cycles in the planarized intersection graph may not correspond to simple cycles in the intersection graph – see Figure 4).

We propose another recursive algorithm, which is also based on our separator lemma but proceeds differently: on one hand, we have to work with intersection graphs instead of clustered planar graphs, and also have to deal with the issue of ensuring simplicity of walks; on the other hand, Corollary 7 is available as a subroutine to compute distances, so fewer input arguments are needed in the new recursive algorithm (notably, we don't need an additional input set B).



Figure 4: These are not simple cycles in the intersection graph!

Theorem 8.
Given a set A of n line segments in $\mathbb{R}^2$, we can compute the girth g_G of its intersection graph G of A in $O(n^{1.483})$ expected time, under the promise that the girth is more than 4.

Proof.
It is known that any intersection graph of n line segments with girth more than 4 must have $O(n)$ edges [42].

Step 1: Build a separator.
We first construct the planarized intersection graph $H = (V_H, E_H)$, as in the proof of Corollary 7, in $\widetilde{O}(n)$ time. Note that $\mu = |V_H| = O(n)$. Define the weight of a point $p \in V_H$ to be 1 if it is the left endpoint of a segment, and 0 otherwise; the total weight is n. We can apply Lemma 2 (with unit loads), for a value p to be set later, to partition V_H into V_1, V_2, S_H such that

- V_i has weight at most $3n/4$ for each $i \in \{1, 2\}$;
- there are no edges of H in $V_1 \times V_2$;
- S_H can be further partitioned into P_H and Q_H such that P_H is the union of at most two paths in H of length $O(p)$, and $|Q_H| = O(n/p)$.

For each $i \in \{1, 2\}$, let A_i consist of the segments $s \in A$ for which all points of V_H on s are in V_i. Note that $|A_i| \leq 3n/4$. Let P consist of the segments with at least one point in P_H. Let Q consist of the segments with at least one point in Q_H and no points in P_H. Let $S = P \cup Q$. Note

that P is the union of at most two walks of length $O(p)$ in G (since any path in H corresponds to a walk in G).

Step 2: Recurse.

For each $i \in \{1, 2\}$, we recursively solve the problem for $A_i \cup P$ to obtain the girth g_{G_i} of the intersection graph G_i of $A_i \cup P$. (Note that the assumption that G has girth more than 4 implies that G_i also has girth more than 4.)

Let $n_i = |A_i \cup P|$. Then $n_1, n_2 \leq 3n/4 + O(p)$ and $n_1 + n_2 \leq n + O(p)$.

Step 3: Compute $\delta_{G_i}[P, P]$.

For each $i \in \{1, 2\}$, we compute $\delta_{G_i}[P, P]$ by Corollary 7 in $\widetilde{O}(n^{3/2-\alpha} + n^{(1/2+\alpha)(3+\omega)/2} + p^{(3+\omega)/2} + p^2 n^{1/2-\alpha})$ expected time.

Step 4: Compute the girth g_G.

We first run BFS from each segment of Q to find the length g_Q of the shortest cycle visiting Q in G. This part takes $\widetilde{O}(n^2/p)$ time.

Next, we compute the length g_{legal} of the shortest legal alternating closed walk where the segments in A_1 are red, the segments in A_2 are blue, and the segments in P are white (discarding the segments in Q). By Lemma 4, the girth $g_{G[A_1 \cup A_2 \cup P]}$ of $G[A_1 \cup A_2 \cup P]$ is equal to $\min\{g_{\text{legal}}, g_{G_1}, g_{G_2}\}$. The final answer is $g_G = \min\{g_{G[A_1 \cup A_2 \cup P]}, g_Q\}$.

To compute g_{legal}, for each $i \in \{1, 2\}$, we first compute the "legal" distance matrices $\delta'_{G_i}[P, P]$: for $u, v \in P$, let

$$\delta'_{G_i}(u, v) = \begin{cases} \delta_{G_i}(u, v) & \text{if } \delta_{G_i}(u, v) \neq \delta_{G[P]}(u, v), \\ +\infty & \text{otherwise.} \end{cases}$$

We already know the matrix $\delta_{G_i}[P, P]$. We can compute $\delta_{G[P]}$ naively in $\widetilde{O}(p^\omega)$ time. Consequently, we know the matrix $\delta'_{G_i}[P, P]$.

Next, we compute

$$\delta''_{G_i}[P, P] = \delta'_{G_i}[P, P] \star \delta_{G[P]}.$$

Although $\delta'_{G_i}[P, P]$ may not satisfy a bounded-difference property, the rows of $\delta_{G[P]}$ satisfy the bounded-difference property, after dividing the matrix into two parts and reordering the columns, since P is the union of at most two walks. So, we can compute $\delta''_{G_i}[P, P]$ in $\widetilde{O}(p^{(3+\omega)/2})$ time by Lemma 1. It also follows that the rows of $\delta''_{G_i}[P, P]$ satisfy the bounded-difference property. Finally, let

$$Z_k = \bigvee_{i=1}^{k} (\delta''_{G_1}[P, P] \star \delta''_{G_2}[P, P])^i.$$

From the definition of legal alternating walks, we see that $Z_n(u, u)$ is precisely the length of the shortest legal alternating closed walk starting at $u \in P$. Thus, g_{legal} is the smallest diagonal entry in Z_n. We can compute Z_n recursively by the following formula:

$$Z_k = (Z_{k/2} \star Z_{k/2}) \vee (\delta''_{G_1}[P, P] \star \delta''_{G_2}[P, P]).$$

(This assumes that k is even, and the odd case is similar.) This requires $O(\log n)$ min-plus matrix products, and all intermediate matrices satisfy the bounded-difference property. So, Z_n can be computed in $\widetilde{O}(p^{(3+\omega)/2})$ time.

Running time.

The time complexity $T(n)$ of the algorithm satisfies the following recurrence:

$$T(n) \leq \max_{\substack{n_1,n_2:\\ n_1,n_2\leq 3n/4+O(p),\\ n_1+n_2\leq n+O(p)}} \Big(T(n_1)+T(n_2)+\widetilde{O}(n^2/p+ n^{3/2-\alpha}+n^{(1/2+\alpha)(3+\omega)/2}+p^{(3+\omega)/2}+p^2n^{1/2-\alpha})\Big).$$

To balance the three terms n^2/p, $n^{(1/2+\alpha)(3+\omega)/2}$, and $p^2n^{1/2-\alpha}$, we set $p=n^{1/2+\rho}$ and $\alpha=3\rho$, with $\rho=\frac{3-\omega}{22+6\omega}>0.0173$, using the latest bound $\omega<2.371339$ [2]. Then the recurrence becomes:

$$T(n) \leq \max_{\substack{n_1,n_2:\ n_1,n_2\leq 3n/4+o(n),\\ n_1+n_2\leq n+o(n)}} \big(T(n_1)+T(n_2)+\widetilde{O}(n^{3/2-\rho})\big),$$

which solves to $T(n)=\widetilde{O}(n^{3/2-\rho})\leq O(n^{1.483})$. ◀

Finally, we remove the assumption that girth is more than 4:

Corollary 9.

Given n line segments in $\mathbb{R}^2$, we can compute the girth of the intersection graph in $O(n^{1.483})$ expected time.

Proof.

We can check whether the intersection graph G contains a 3-cycle in $O(n^{1.408})$ time by an algorithm of Chan [14, Theorem 5.1], or a 4-cycle in $\widetilde{O}(n)$ time by another algorithm of Chan [14, Theorem 6.2]. So, we may assume that the girth is greater than 4, and the result follows from Theorem 8. ◀

In the full version, we describe how to modify our algorithms more generally for arbitrary algebraic curves or semialgebraic sets that are connected and have constant description complexity.

Our results are unlikely to be optimal. Although we do not know how to improve Corollary 7, we are currently working on ideas that might further improve the time bound of Corollary 9 for the case of line segments, by exploiting special properties about intersection graphs with large girth.

References

[1] Pankaj K. Agarwal and Marc J. van Kreveld. Connected component and simple polygon intersection searching. Algorithmica, 15(6):626–660, 1996. doi:10.1007/BF01940884.

[2] Josh Alman, Ran Duan, Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. More asymmetry

yields faster matrix multiplication. In Proc. 36th ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 2005–2039, 2025. doi:10.1137/1.9781611978322.63.

[3] Noga Alon, Raphael Yuster, and Uri Zwick. Color-coding. J. ACM, 42(4):844–856, 1995. doi:10.1145/210332.210337.

[4] Jon Louis Bentley and Thomas Ottmann. Algorithms for reporting and counting geometric intersections. IEEE Trans. Computers, 28(9):643–647, 1979. doi:10.1109/TC.1979.1675432.

[5] Béla Bollobás. Chromatic number, girth and maximal degree. Discret. Math., 24(3):311–314, 1978. doi:10.1016/0012-365X(78)90102-4.

[6] Édouard Bonnet, Sergio Cabello, and Wolfgang Mulzer. Maximum matchings in geometric intersection graphs. Discret. Comput. Geom., 70(3):550–579, 2023. doi:10.1007/S00454-023-00564-3.

[7] Karl Bringmann and Alejandro Cassis. Faster knapsack algorithms via bounded monotone min-plus-convolution. In Proc. 49th International Colloquium on Automata, Languages, and Programming (ICALP), volume 229 of LIPIcs, pages 31:1–31:21, 2022. doi:10.4230/LIPIcs.ICALP.2022.31.

[8] Karl Bringmann, Anita Dürr, and Adam Polak. Even faster knapsack via rectangular monotone min-plus convolution and balancing. In Proc. 32nd European Symposium on Algorithms (ESA), volume 308 of LIPIcs, pages 33:1–33:15, 2024. doi:10.4230/LIPIcs.ESA.2024.33.

[9] Karl Bringmann, Fabrizio Grandoni, Barna Saha, and Virginia Vassilevska Williams. Truly subcubic algorithms for language edit distance and RNA folding via fast bounded-difference min-plus product. SIAM J. Comput., 48(2):481–512, 2019. doi:10.1137/17M112720X.

[10] Karl Bringmann, Sándor Kisfaludi-Bak, Marvin Künnemann, André Nusser, and Zahra Parsaeian. Towards subquadratic diameter computation in geometric intersection graphs. In Proc. 38th International Symposium on Computational Geometry (SoCG), volume 224 of LIPIcs, pages 21:1–21:16, 2022. doi:10.4230/LIPIcs.SOCG.2022.21.

[11] Sergio Cabello. Subquadratic algorithms for the diameter and the sum of pairwise distances in planar graphs. ACM Trans. Algorithms, 15(2):21:1–21:38, 2019. doi:10.1145/3218821.

[12] Sergio Cabello and Wolfgang Mulzer. Minimum cuts in geometric intersection graphs. Comput. Geom., 94:101720, 2021. doi:10.1016/J.COMGEO.2020.101720.

[13] Parinya Chalermsook, Jittat Fakcharoenphol, and Danupon Nanongkai. A deterministic near-linear time algorithm for finding minimum cuts in planar graphs. In Proc. 15th ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 828–829, 2004. doi:10.5555/982792.982916.

[14] Timothy M. Chan. Finding triangles and other small subgraphs in geometric intersection graphs. In Proc. 34th ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 1777–1805, 2023. doi:10.1137/1.9781611977554.CH68.

[15] Timothy M. Chan, Hsien-Chih Chang, Jie Gao, Sándor Kisfaludi-Bak, Hung Le, and Da Wei Zheng. Truly subquadratic time algorithms for diameter and related problems in graphs of bounded VC-dimension. CoRR, abs/2510.16346, 2025. To appear in FOCS 2025. doi:10.48550/arXiv.2510.16346.

[16] Timothy M. Chan and Moshe Lewenstein. Clustered integer 3SUM via additive combinatorics. In Proc. 47th ACM Symposium on Theory of Computing (STOC), pages 31–40, 2015. doi:10.1145/2746539.2746568.

[17] Timothy M. Chan and Dimitrios Skrepetos. All-pairs shortest paths in geometric intersection graphs. J. Comput. Geom., 10(1):27–41, 2019. doi:10.20382/JOCG.V10I1A2.

[18] L. Sunil Chandran and C. R. Subramanian. Girth and treewidth. J. Comb. Theory B, 93(1):23–32, 2005. doi:10.1016/J.JCTB.2004.05.004.

[19] Hsien-Chih Chang and Hsueh-I Lu. Computing the girth of a planar graph in linear time. SIAM J. Comput., 42(3):1077–1094, 2013. doi:10.1137/110832033.

[20] Shucheng Chi, Ran Duan, and Tianle Xie. Faster algorithms for bounded-difference min-plus product. In Proc. 33th ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 1435–1447, 2022. doi:10.1137/1.9781611977073.60.

[21] Shucheng Chi, Ran Duan, Tianle Xie, and Tianyi Zhang. Faster min-plus product for monotone instances. In Proc. 54th ACM Symposium on Theory of Computing (STOC), pages 1529–1542, 2022. doi:10.1145/3519935.3520057.

[22] R. J. Cook. Chromatic number and girth. Periodica Mathematica Hungarica, 6(1):103–107, 1975. doi:10.1007/BF02018401.

[23] Reinhard Diestel. Graph Theory, volume 173 of Graduate texts in mathematics. Springer, 2015. doi:10.1007/978-3-662-70107-2.

[24] Hristo N. Djidjev. Computing the girth of a planar graph. In Proc. 27th International Colloquium on Automata, Languages and Programming (ICALP), volume 1853 of LIPIcs, pages 821–831, 2000. doi:10.1007/3-540-45022-X_69.

[25] Hristo N. Djidjev. A faster algorithm for computing the girth of planar and bounded genus graphs. ACM Trans. Algorithms, 7(1):3:1–3:16, 2010. doi:10.1145/1868237.1868240.

[26] Guillaume Ducoffe, Michel Habib, and Laurent Viennot. Diameter, eccentricities and distance oracle computations on H-minor free graphs and graphs of bounded (distance) Vapnik-Chervonenkis dimension. SIAM J. Comput., 51(5):1506–1534, 2022. doi:10.1137/20M136551X.

[27] Paul Erdös. Graph theory and probability. Canadian Journal of Mathematics, 11:34–38, 1959. doi:10.1007/978-0-8176-4842-8_19.

[28] Jeff Erickson. Planar separators. Notes, 2023. URL: https://jeffe.cs.illinois.edu/teaching/comptop/2023/notes/14-planar-separators.pdf.

[29] Jittat Fakcharoenphol and Satish Rao. Planar graphs, negative weight edges, shortest paths, and near linear time. J. Comput. Syst. Sci., 72(5):868–889, 2006. doi:10.1016/J.JCSS.2005.05.007.

[30] Pawel Gawrychowski, Haim Kaplan, Shay Mozes, Micha Sharir, and Oren Weimann. Voronoi diagrams on planar graphs, and computing the diameter in deterministic $\tilde{O}(n^{5/3})$ time. SIAM J. Comput., 50(2):509–554, 2021. doi:10.1137/18M1193402.

[31] Yuzhou Gu, Adam Polak, Virginia Vassilevska Williams, and Yinzhan Xu. Faster monotone min-plus product, range mode, and single source replacement paths. In Proc. 48th International Colloquium on Automata, Languages, and Programming (ICALP), volume 198 of LIPIcs, pages 75:1–75:20, 2021. doi:10.4230/LIPIcs.ICALP.2021.75.

[32] Alon Itai and Michael Rodeh. Finding a minimum circuit in a graph. SIAM J. Comput., 7(4):413–423, 1978. doi:10.1137/0207033.

[33] Haim Kaplan, Katharina Klost, Wolfgang Mulzer, Liam Roditty, Paul Seiferth, and Micha Sharir. Triangles and girth in disk graphs and transmission graphs. In Proc. 27th European Symposium on Algorithms (ESA), volume 144 of LIPIcs, pages 64:1–64:14, 2019. doi:10.4230/LIPIcs.ESA.2019.64.

[34] Philip Klein and Shay Mozes. Optimization algorithms for planar graphs. Book draft, 2024. URL: https://planarity.org.

[35] Katharina Klost and Wolfgang Mulzer. Robust algorithms for finding triangles and computing the girth in unit disk and transmission graphs. CoRR, abs/2405.01180, 2024. doi:10.48550/arXiv.2405.01180.

[36] Hung Le and Christian Wulff-Nilsen. VC set systems in minor-free (di)graphs and applications. In Proc. 35th ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 5332–5360, 2024. doi:10.1137/1.9781611977912.192.

[37] Jason Li and Merav Parter. Planar diameter via metric compression. In Proc. 51st ACM Symposium on Theory of Computing (STOC), pages 152–163, 2019. doi:10.1145/3313276.3316358.

[38] Richard J. Lipton and Robert Endre Tarjan. A separator theorem for planar graphs. SIAM Journal of Applied Mathematics, 36(2):177–189, 1979. doi:10.1137/0136016.

[39] László Lovász. On chromatic number of finite set-systems. Acta Mathematica Academiae Scientiarum Hungarica, 19(1):59–67, 1968. doi:10.1007/BF01894680.

[40] Xiao Mao. Breaking the cubic barrier for (unweighted) tree edit distance. In Proc. 62nd IEEE Symposium on Foundations of Computer Science (FOCS), pages 792–803, 2021. doi:10.1109/FOCS52979.2021.00082.

[41] Gary L. Miller. Finding small simple cycle separators for 2-connected planar graphs. J. Comput. Syst. Sci., 32(3):265–279, 1986. doi:10.1016/0022-0000(86)90030-9.

[42] János Pach and Micha Sharir. On planar intersection graphs with forbidden subgraphs. J. Graph Theory, 59(3):205–214, 2008. doi:10.1002/JGT.20332.

[43] Barna Saha and Christopher Ye. Faster approximate all pairs shortest paths. In Proc. 35th ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 4758–4827, 2024. doi:10.1137/1.9781611977912.170.

[44] Jeffrey D. Ullman and Mihalis Yannakakis. High-probability parallel transitive-closure algorithms. SIAM J. Comput., 20(1):100–125, 1991. doi:10.1137/0220006.

[45] Virginia Vassilevska Williams. On some fine-grained questions in algorithms and complexity. In Proc. International Congress of Mathematicians (ICM), pages 3447–3487, 2018. doi:10.1142/9789813272880_0188.

[46] Oren Weimann and Raphael Yuster. Computing the girth of a planar graph in $O(n \log n)$ time. SIAM J. Discret. Math., 24(2):609–616, 2010. doi:10.1137/090767868.

[47] Uri Zwick. All pairs shortest paths using bridging sets and rectangular matrix multiplication. J. ACM, 49(3):289–317, 2002. doi:10.1145/567112.567114.